

计算机体系结构

13. 缓存一致性

李建华

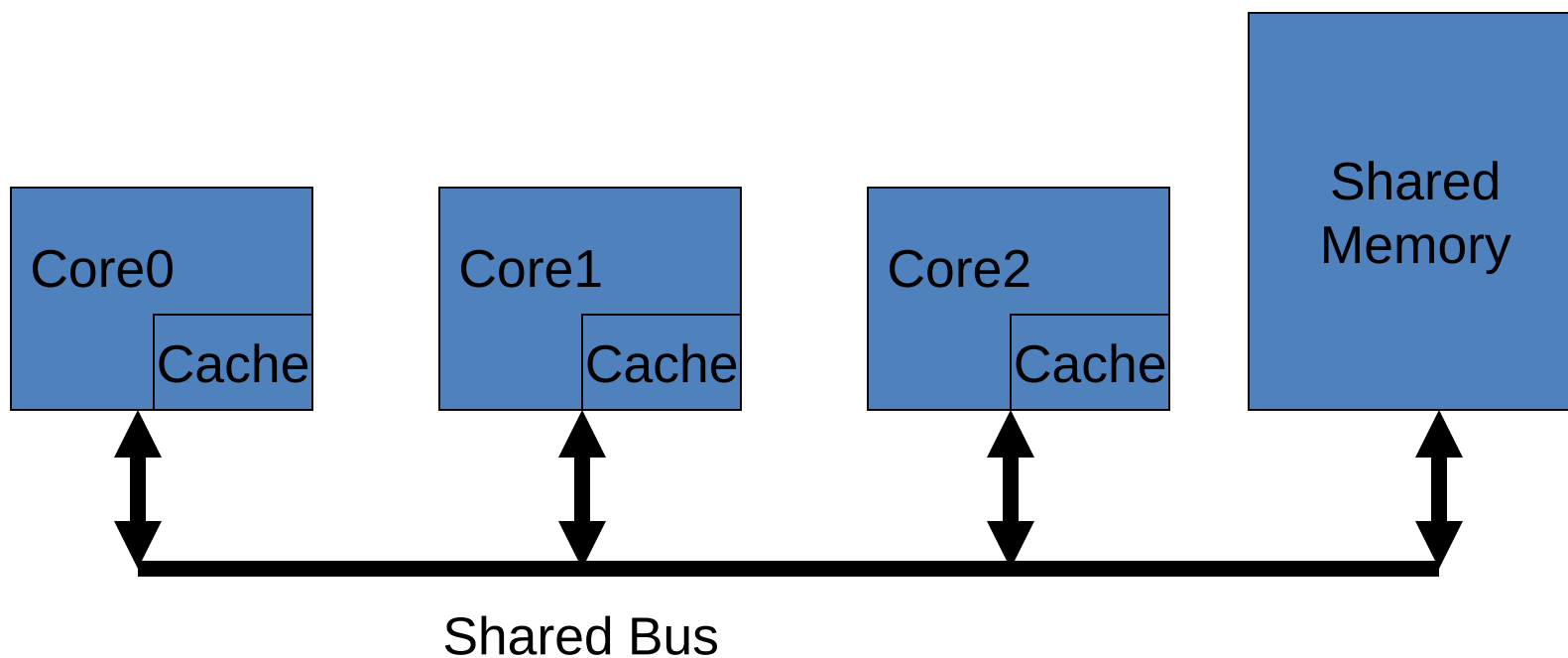
计算机与信息学院

The contents of the slides are adapted from CA course of wisc, princeton, mit, berkeley, edinburgh, and eth.
The uses of the slides of this course are for educational purposes only and should be used only in conjunction with the textbook. Derivatives of the slides must acknowledge the copyright notices of this and the originals.

前 瞻

- 我们已经学习了多核处理器架构
- 现在，我们来看下典型的多核架构是什么
- 二种基于共享存储器的通信架构：
 - 基于总线的共享存储器机器 (small-scale)
 - 基于片上网络的共享存储器机器 (large-scale)
 - 若时间允许，我们会讨论片上网络 (NoC) .

基于总线的共享存储器



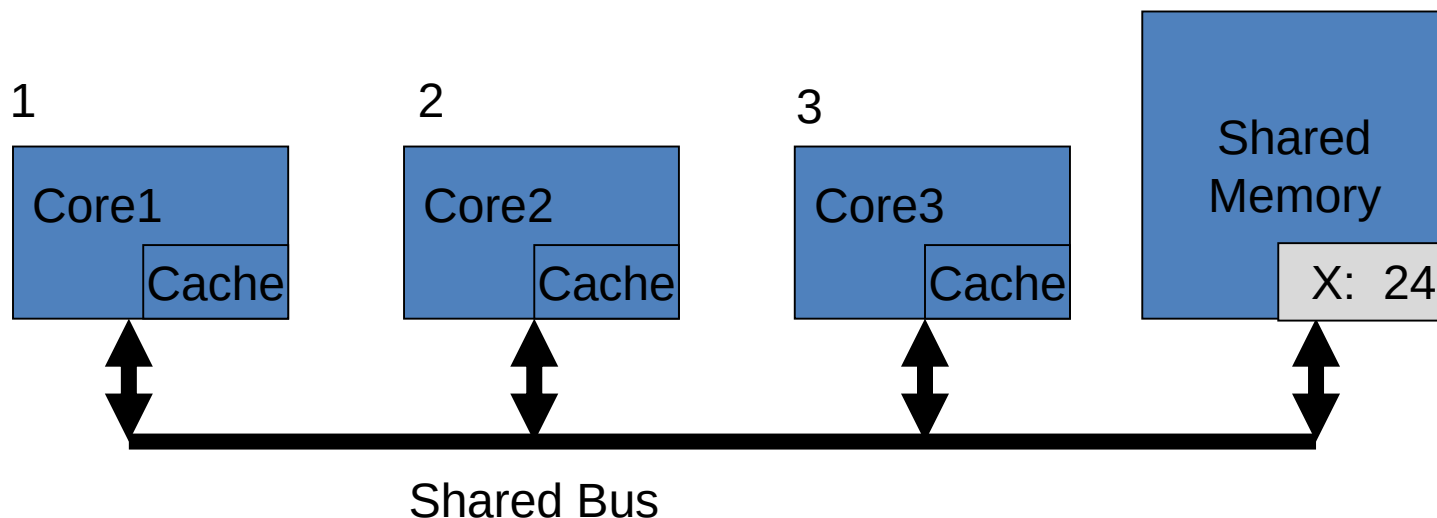
组织

- 总线是简单的物理连接方式(**A bundle of wires**)
- 总线的带宽会**限制**其连接的总的**设备数量**
 - 总线由所有的设备共享
- 从现在开始，我们假定：**每个CPU/core仅仅具备一级缓存结构。**
 - 该假设的目的是简化后续一致性协议的讨论

缓存一致性问题

- 假设只有一级缓存，其下一级就是主存。
- 每个 core 在其私有的缓存里进行写操作
- 其它core的缓存可能保留有共享的缓存块
- **读写操作可能使得有些共享的块过期**（某些缓存保存的是旧的数据）
- 此时，仅仅更新主存是不够的。
 - 缓存不管采取write-back 还是 write-through 策略都不能解决问题

示例



- Core1 读 X: 从memory中获取24, 并进行本地缓存;
- Core2 读 X: 从memory中获取24, 并进行本地缓存;
- Core1 写 32 到 X: 更新其本地缓存的X副本
- Core3 读 X: 它读取到的值是什么?

Memory 和 Core2 认为 X 是 24
Core1 认为 X 是 32



注意：这里采用写直达缓存是不足以确保系统的一致性的。

解决方案: Cache Coherence Protocol



MORGAN & CLAYPOOL PUBLISHERS

A Primer on Memory Consistency and Cache Coherence *Second Edition*

Vijay Nagarajan
Daniel J. Sorin
Mark D. Hill
David A. Wood

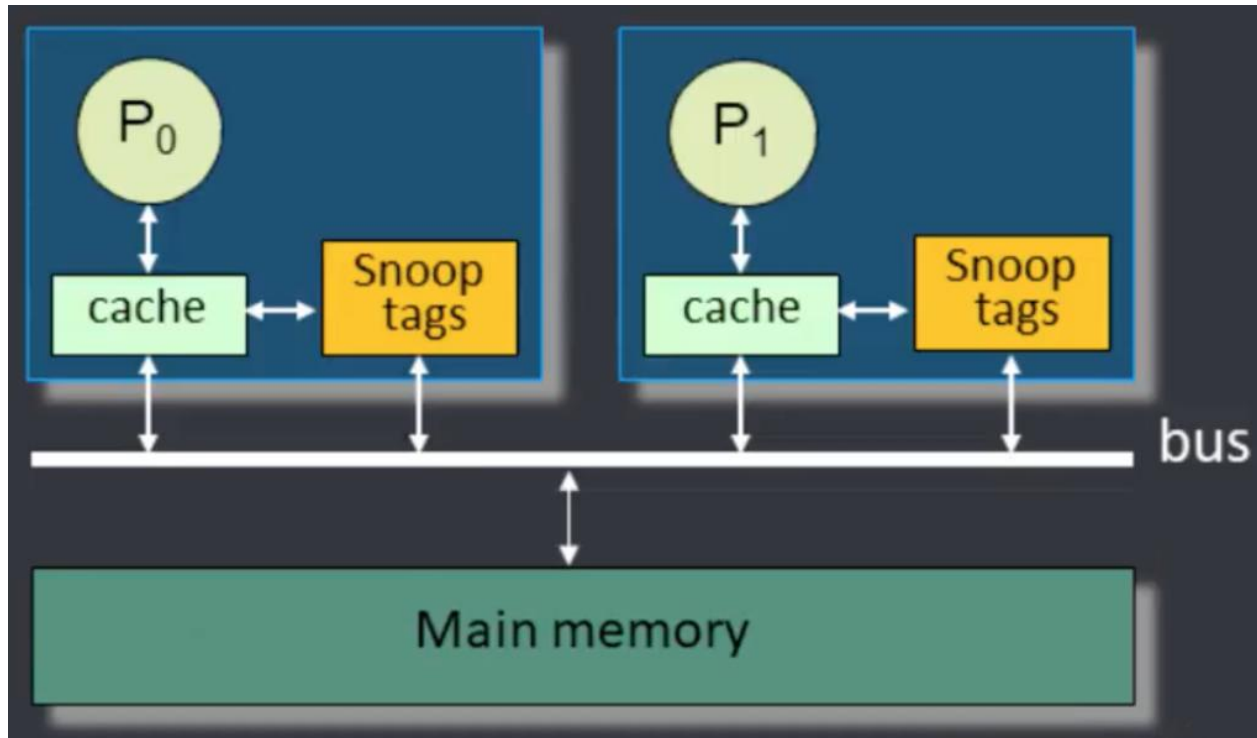
6	Coherence Protocols	91
6.1	The Big Picture	91
6.2	Specifying Coherence Protocols	93
6.3	Example of a Simple Coherence Protocol	94
6.4	Overview of Coherence Protocol Design Space	96
6.4.1	States	97
6.4.2	Transactions	101
6.4.3	Major Protocol Design Options	103
6.5	References	105
7	Snooping Coherence Protocols	107
7.1	Introduction to Snooping	107
7.2	Baseline Snooping Protocol	111
7.2.1	High-Level Protocol Specification	112
7.2.2	Simple Snooping System Model: Atomic Requests, Atomic Transactions	113
7.2.3	Baseline Snooping System Model: Non-Atomic Requests, Atomic Transactions	117
8	Directory Coherence Protocols	151
8.1	Introduction to Directory Protocols	151
8.2	Baseline Directory System	153
8.2.1	Directory System Model	153
8.2.2	High-Level Protocol Specification	153
8.2.3	Avoiding Deadlock	155
8.2.4	Detailed Protocol Specification	158
8.2.5	Protocol Operation	159
8.2.6	Protocol Simplifications	161
8.3	Adding the Exclusive State	162
8.3.1	High-Level Protocol Specification	162
8.3.2	Detailed Protocol Specification	164
8.4	Adding the Owned State	164
8.4.1	High-Level Protocol Specification	164

Cache Coherence Protocol

- 实现一个让每个core知道当前每个block的最新值的方案是非常复杂的
- 在snoop coherence protocol下面:
 - 每个Core (cache system)在总线上 ‘**snoops**’ (i.e. **watches continually**) 它感兴趣的 (本地缓存有副本) 数据地址的写操作 (**write activity**)
- Snooping protocol需要一个全局的总线来支撑, master的所有通信能被所有的slaves看到, 而且写的顺序要一致。
- 一个可扩展性更好的方案是: directory coherence protocols

Snoop coherence protocol

- Snoop Coherence Protocol 的基础架构



This figure is from: <http://thebeardsage.com/cache-coherence-protocols/>

Snooping Protocol 的实现

- 写无效 (Write Invalidate)
 - 若core 想写一个block -> 需要获取一个总线周期, 并发送 'write invalidate' 消息
 - 所有的snooping caches看到消息后, 会 invalidate 自己的副本 (可能的副本) ;
 - 收集到所有的 **acknowledgement** 后, core 可以写目标block。(假定该写操作会直达到memory)
 - 后续所有其它core对该block的读取均会miss, 从而会重新获取block。[此刻的多核系统是一致的]

<https://www.scss.tcd.ie/Jeremy.Jones/VivioJS/caches/MESIHelp.htm>

Snooping Protocol 的实现

- 写更新 (Write Update)
 - 若core 想写一个block -> 需要获取一个总线周期, 并将要写的数据通过总线进行广播
 - 所有的snooping caches 通过总线更新其副本
- 注意: 无论是写无效还是写更新, 同时发生的写操作可以通过总线的仲裁机制来避免。
 - 在任何时刻, 只有一个core可以是总线的master。
 - 因此, 同时写操作的冲突不会造成问题。

更新 or 无效?

- 写更新看起来最简单、直接、速度快, 却会消耗大量网络带宽:
 - 对同一个word的多个写操作 (中间没有读操作) 只需要一个 “write invalidate” 消息, 但在写更新下每个写都需要一次更新 (更多的traffic)
 - 对同一个缓存块的多个写 (multi-word cache block) 只需要一个 “write invalidate” 消息, 但在写更新下每个写都需要一次更新 (更多的traffic)
 - **所以, 写更新的开销更大。**

更新 or 无效?

- 由于缓存中的spatial和temporal局部性，对同一个word以及同一个block的多个写经常发生。
- 此外，总线带宽在共享处理器的多核系统中是一个稀缺资源。
- 经验表明，写无效机制会使用相当少的总线带宽资源。
- 当前，大多数的产品采取write invalidate方案。
- 下面，我们仅讨论 write invalidate方案的实现细节。

如何实现 coherence?

- 为了实现一致性, 知道一个block**没有**被共享 (在其它cache里没有副本) 可以避免发送消息 -> **Reduce Network traffic**
- 写无效协议假定block的更新值会被**written back** 到Memory中
 - 在这种情况下, 其它的core发生读缺失时可能会获得该block的旧值 (**re-fetch from the memory**)。
- 因此, 需要一个协议来处理上述情况。
- 有很多Coherence Protocols, 请参考: [Sorin's book](#).

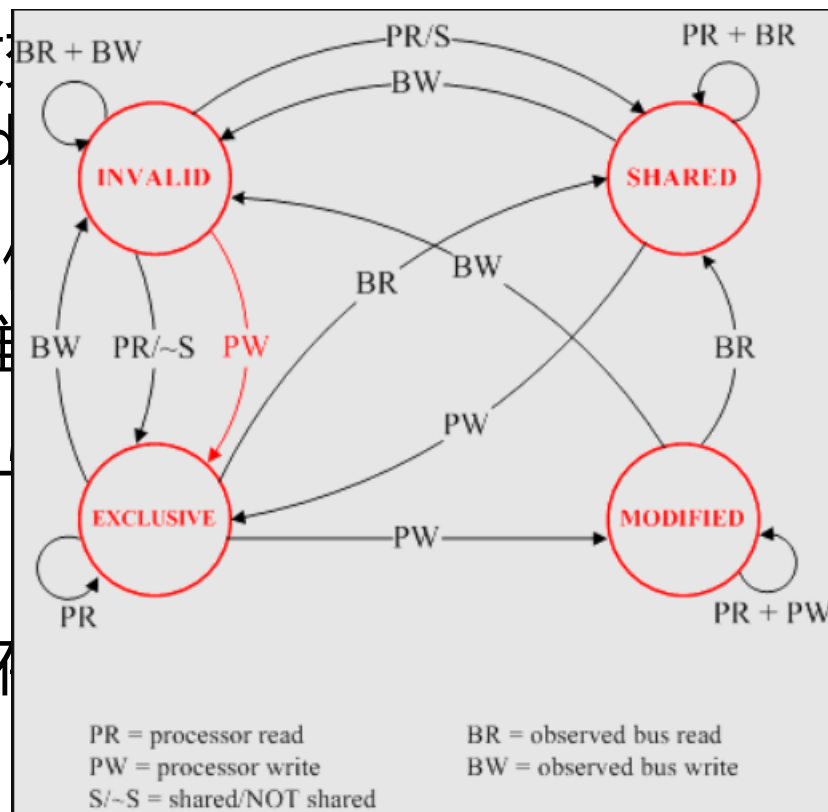
示例: MESI 协议

- MESI是一个用于多核处理器的基于write invalidate 的协议，旨在最小化总线的使用。
- 允许缓存使用 ‘**write back**’ 方案 - i.e. 主存直到 ‘dirty’ cache line 从缓存中踢出时才被更新。
- 要实现上述功能: 需要扩展已有的cache tag内容, i.e. valid bit, ‘dirty’ bit, + **coherence state bits**.
 - 需要添加额外的一致性状态信息
 - 与原来的辅助信息一起，保存在tag store中。

示例: MESI 协议

- 每个缓存块处于4个状态之一 (需要用2bits编码)

- **Modified** – 缓存块已经改
当前唯一的缓存副本。(其d
- **Exclusive** – 缓存块当前的
该块是当前所有缓存中的唯
- **Shared** – 缓存块当前的值
块在缓存中存在多个副本。
- **Invalid** – 该块当前没有保
cache)



<https://www.scss.tcd.ie/Jeremy.Jones/VivioJS/caches/MESIHelp.htm>

示例: MESI 协议

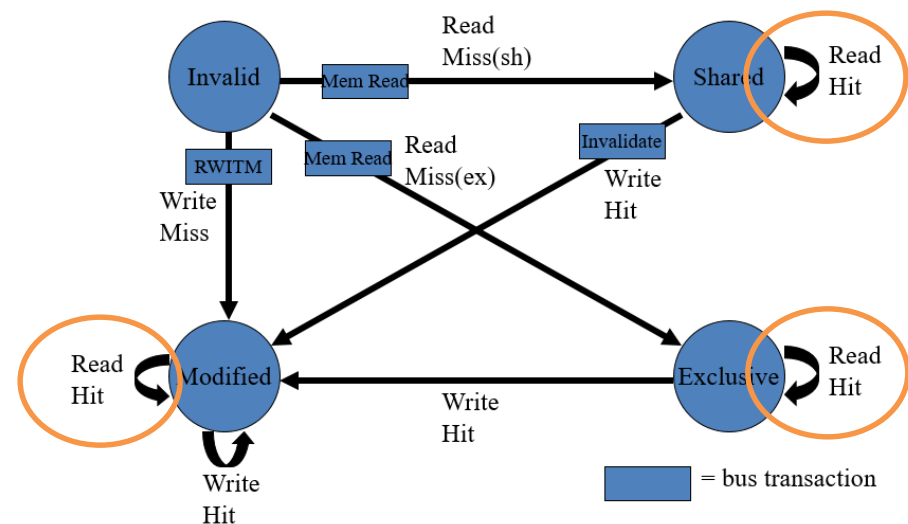
- 缓存块状态的改变是存储读写操作（事件）的一个函数。
 - 也就是FSM — 有限状态机
- 上面的事件可能是：
 - 本地core的读写操作 (i.e. 缓存访问)
 - 总线操作 – 来自其它core的操作
- 仅当操作的地址与缓存块的地址匹配，该缓存块的状态才受到影响。
 - 本地操作的**目标地址** 或者 总线操作的**目标地址**
 - 本地缓存中保存了与目标地址指向的缓存块

示例: MESI 协议

- 协议的具体操作可以用缓存块在发生以下事件时的状态转换图来进行描述：
 - Read Hit
 - Read Miss
 - Write Hit
 - Write Miss
- 将所有的状态和事件的转换关系整合到一起，就是整个协议的状态图。
- 下面，对状态转换进行分情况解析。

MESI – 本地 Read Hit

- 命中的缓存行 (line) 的状态为MES之一
- 该缓存行保存有正确的值 (若该块处在M状态, 那么它保存有唯一正确的值。)
- 只需要将保存的值返回给本地的请求者
- 该块没有发生状态转换 (状态保持不变)



MESI – 本地 Read Miss (1)

① 其它缓存中没有副本

- Core 在总线上向memory发送一个请求
- 值返回给该core之后，保存在局部缓存，状态置为：E

② 其中一个缓存有一个处于E状态的副本

- Core在总线上向memory发一个请求
- 侦听的缓存（具有E副本）看到后，将值放到总线上
- Memory访问请求被取消
- Core获得了值，并进行在本地缓存进行保存
- 2个具有相同值的缓存块都切换到S状态

MESI – 本地 Read Miss (2)

③ 几个缓存具有S状态的副本

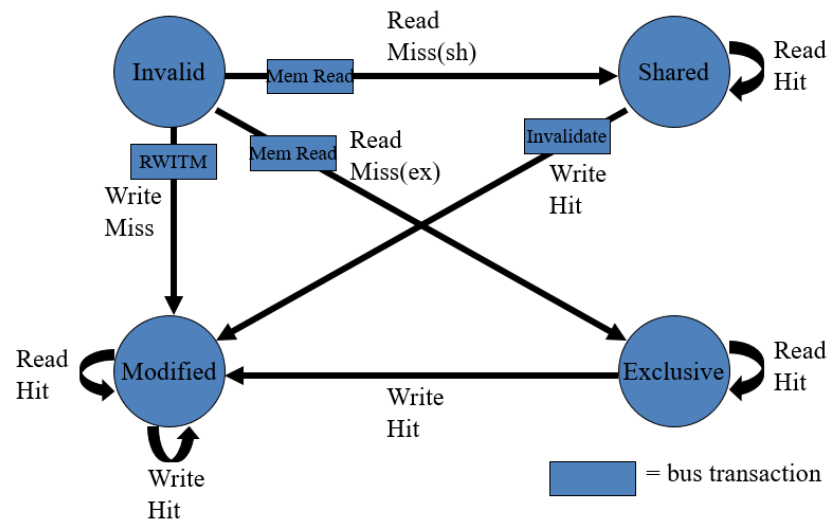
- Core在总线上向memory发一个请求
- 其中一个侦听的缓存（具有S副本）看到后，将值放到总线上 (**arbitrated**)
- Memory访问请求被取消
- Core获得了值，并进行在本地缓存进行保存
- 本地的缓存块切换到S状态
- 其它副本的状态维持S不变

MESI – 本地 Read Miss (3)

- ④ 其中一个缓存有一个处于M状态的副本
 - Core在总线上向memory发一个请求
 - 侦听的缓存（具有E副本）看到后，将值放到总线上
 - Memory访问请求被取消
 - Core获得了值，并进行在本地缓存进行保存
 - 本地的缓存块副本状态置为S
 - 处于E状态的块的值刷回memory
 - 处于E状态的块的状态进行切换: M -> S

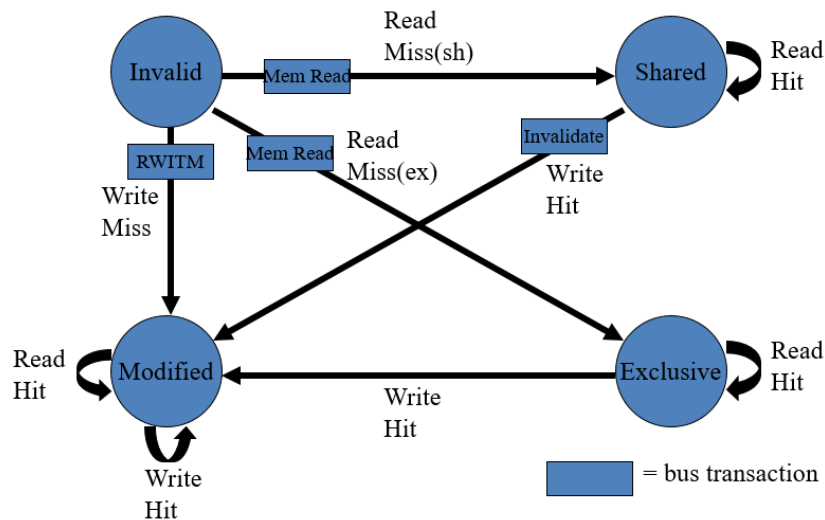
MESI – 本地 Write Hit (1)

- 命中的缓存块 (line) 的状态为MES之一
- 若在M状态
 - 只有一个副本, 且处于 'dirty' 状态
 - 直接更新本地的值即可
 - 无需状态转换 (依然是M状态)
- 若在E状态
 - 直接更新本地的值
 - 切换块的状态: E -> M
 - 将dirty bit置位



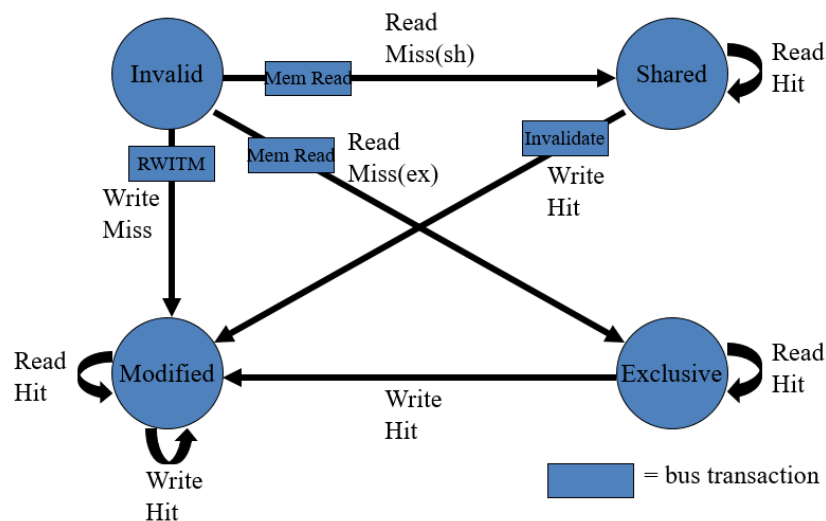
MESI – 本地 Write Hit (2)

- 若在S状态
 - Core在总线上广播一个invalidate消息
 - 具有S状态副本的缓存，会将相应的块置无效：S->I
 - 更新本地缓存的块的值
 - 将该块的状态进行切换：S->M



MESI – 本地 Write Miss (1)

- 具体的操作依赖于其它缓存中副本的状态
- 如果没有其它副本存在：
 - 从Memory中将目标缓存行读取到本地缓存
 - Write allocate
 - 更新缓存行的值（写入新值）
 - 更新缓存行的状态为M



MESI – 本地 Write Miss (2)

- 存在其它副本：一个处于E的副本或者多个处于S的副本
 - 从Memory中将缓存行读取到本地缓存，同时发送一个总线事务RWITM (**read with intent to modify**);
 - 其它缓存的控制器看到这个事务，会将本地的副本置为I 状态（若存在副本的话）；
 - 更新本地缓存行，并将状态置为M。

MESI – 本地 Write Miss (3.1)

- 存在一个处于M状态的副本
- Core发送一个 RWITM 总线事务
- 副本所在的缓存的控制器监听到该事务后：
 - 阻塞 RWITM 请求
 - 接管总线
 - 将其保存的副本写回到Memory
 - 将其状态置为 I

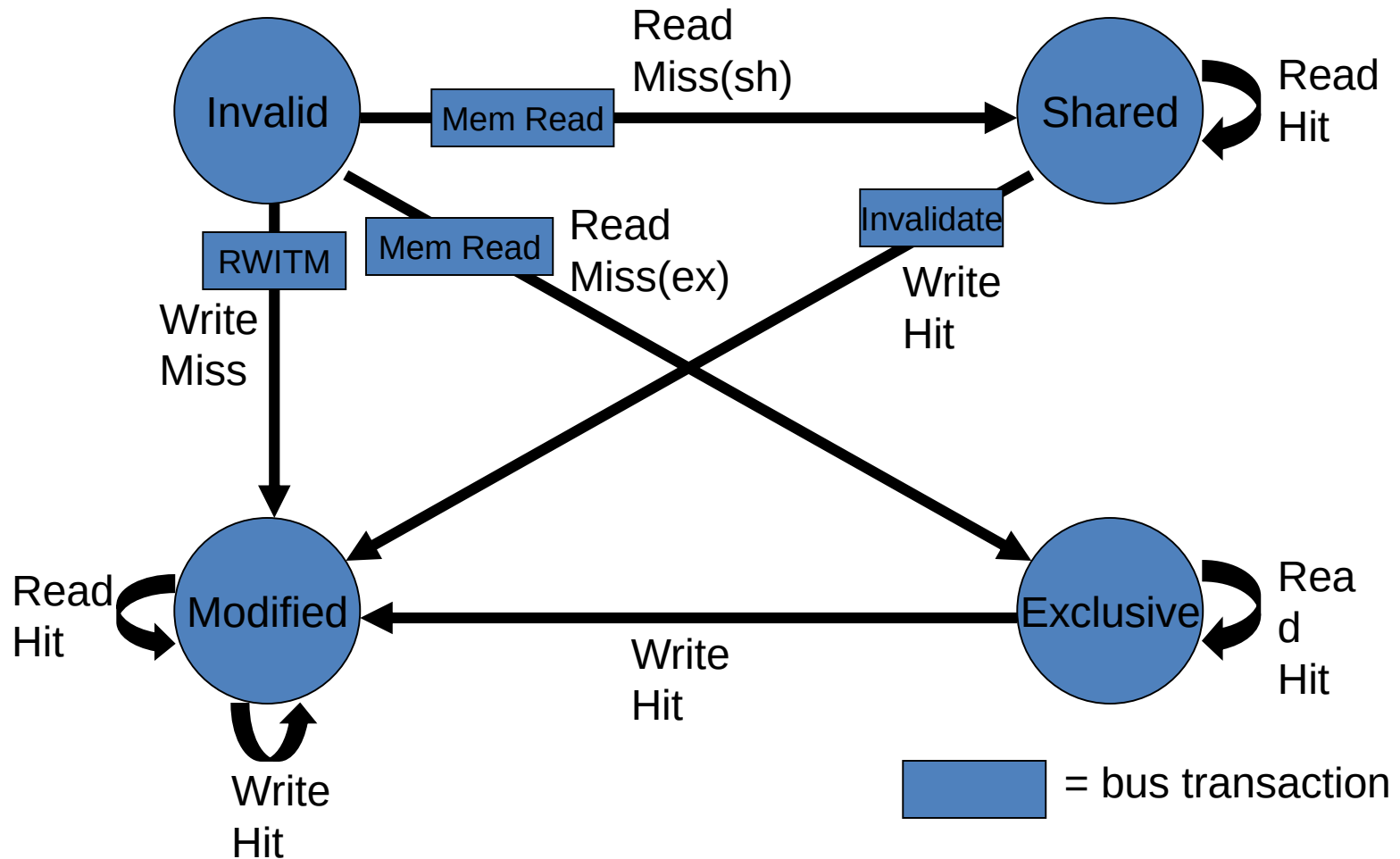
MESI – 本地 Write Miss (3.2)

- 存在一个处于M状态的副本 (绪)
- 原来请求的core, 重发 RWITM 事务请求;
- 当前, 目标缓存行在所有的缓存中没有副本:
 - 从Memory中读取目标缓存行到本地缓存local cache
 - 更新本地缓存行的值 (写入值)
 - 更新本地缓存行的状态为 M

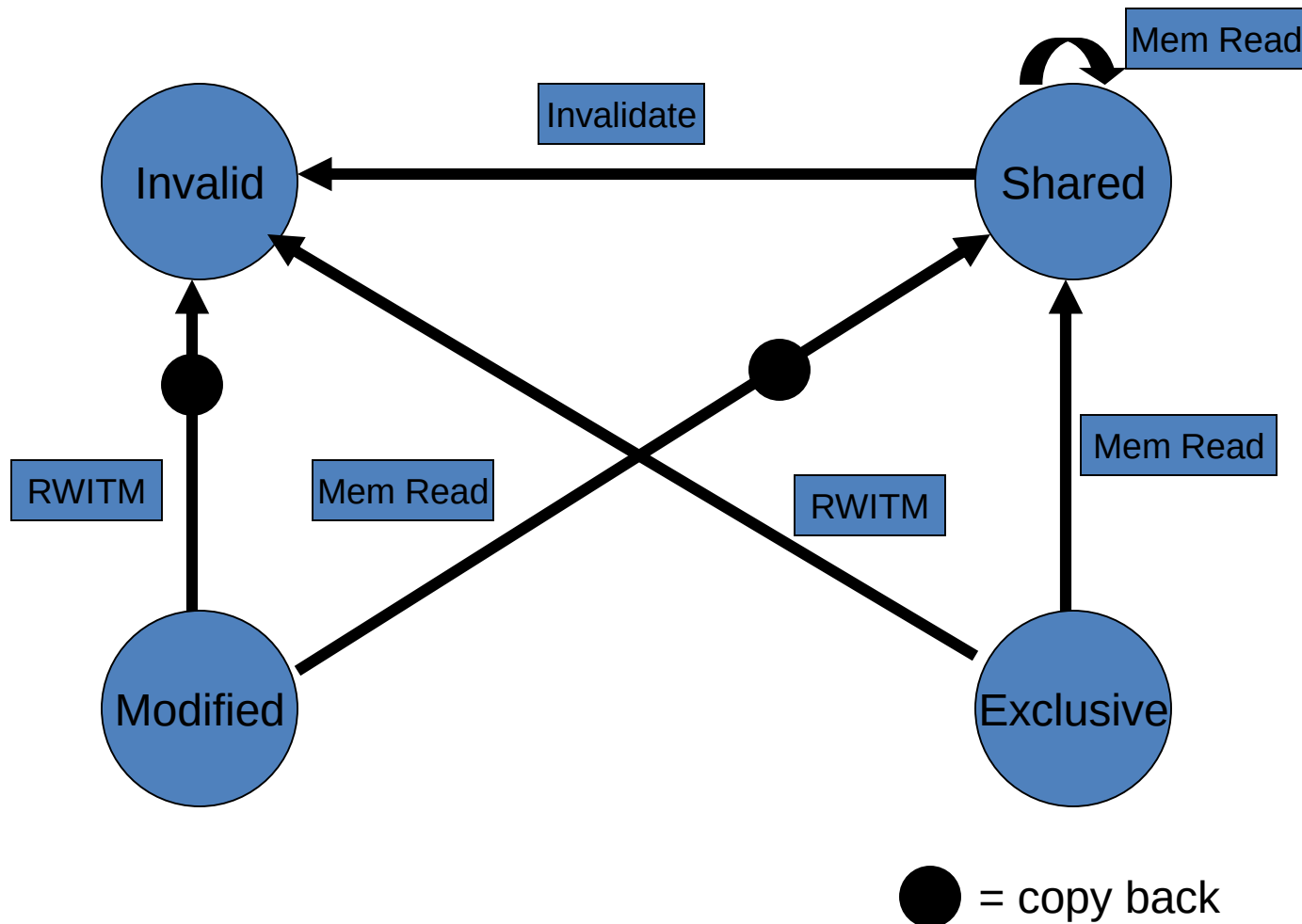
MESI – 总结

- 协议涉及的所有的信息和操作可以用一个状态转换图来紧凑地表示
- 状态转换图展示了当一个core要读写某个缓存行时要发生的操作
 - 可以是来自本地core的操作
 - read hit/miss, write hit/miss
 - 也可以是来自其它core的操作，最终本地core通过侦听总线看到的是总线事务：
 - Mem read, RWITM, Invalidate

MESI – 本地发起的访问



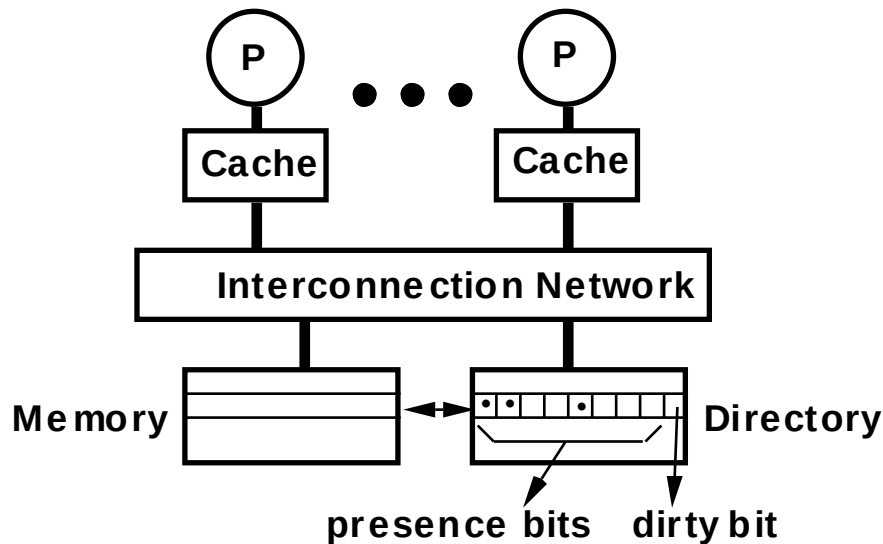
MESI – 远程发起的访问



Directory Coherence

- 由于依赖于总线架构，Snoopy 一致性协议的可扩展性较差 (**BUS is the bottleneck!**)
- Directory一致性协议具有更好的可扩展性
 - 通过记录一个memory block的共享者信息来避免广播，从而可以用点到点的消息来维持一致性。
 - 目录一致性协议更加灵活，可以适用任何可扩展的点到点网络拓扑。
 - 然而，目录一致性协议也存在问题。大家可以思考一下！

基本方案 (Censier & Feautrier)



- Assume "k" processors.
- With each cache-block in memory: k presence-bits, and 1 dirty-bit
- With each cache-block in cache: 1 valid bit, and 1 dirty (owner) bit

- Read from main memory by P-i:
 - If **dirty-bit is OFF** then { read from main memory; turn p[i] ON; }
 - if **dirty-bit is ON** then { recall line from dirty PE (cache state to shared); update memory; turn dirty-bit OFF; turn p[i] ON; supply recalled data to PE-i; }
- Write to main memory:
 - If dirty-bit OFF then { send invalidations to all PEs caching that block; turn dirty-bit ON; turn P[i] ON; ... }
 - ...

Directory Protocol的问题

- Memory和directory带宽的扩展性
 - 不能使用集中的main memory 或者 directory
 - Need a **distributed** memory and directory structure
- Directory memory requirements do not scale well
 - **Number of presence bits grows with number of PEs**
 - Many ways to get around this problem
 - limited pointer schemes of many flavors
 - More research focus on this topic

2025年的课程到此结束

预祝各位同学考试顺利！